

## Experiment No. 1

**Title:** Read and display image, perform common operations on image using OpenCV

- **Input Image for all operation: Lenna.png**



Fig:Lenna.png

### 1(a) Aim:

To create a program to display the image using read and show operation.

- **Apparatus use:**

- 1) A PC with Python 3.7.8 and OpenCV
- 2) Jupyter notebook

- **Theory**

To install these libraries, we need to run these pip commands in cmd:

```
pip install opencv-python  
pip install numpy  
pip install matplotlib
```

#### 1) **cv2.imread():**

To **read** the images **cv2.imread()** method is used. This method loads an image from the specified file. If the image cannot be read (because of missing file, improper permissions, unsupported or invalid format) then this method returns an empty matrix.

**Syntax:** cv2.imread(path, flag)

**Parameters:**

**path:** A string representing the path of the image to be read.

**flag:** It specifies the way in which image should be read. It's default value is cv2.IMREAD\_COLOR

---

**Return Value:** This method returns an image that is loaded from the specified file.

## 2) **cv2.imshow()**

cv2.imshow() method is used to display an image in a window. The window automatically fits the image size.

**Syntax:** cv2.imshow(window\_name, image)

Parameters:

**window\_name:** A string representing the name of the window in which image to be displayed.

**image:** It is the image that is to be displayed.

**Return Value:** It doesn't returns anything.

## 3) **cv2.waitKey()**

waitkey() function of Python OpenCV allows users to display a window for given milliseconds or until any key is pressed. It takes time in milliseconds as a parameter and waits for the given time to destroy the window, if 0 is passed in the argument it waits till any key is pressed.

## 4) **Cv2.destroyAllWindows**

destroyAllWindows() function allows users to destroy or close all windows at any time after exiting the script. If you have multiple windows open at the same time and you want to close then you would use this function. It doesn't take any parameters and doesn't return anything. It is similar to destroyWindow() function but this function only destroys a specific window unlike destroyAllWindows().

### • **Procedure**

1. Open Python 3.7.8 with Opencv
2. Import cv2.
3. Paste a test image in the directory.
4. Create variable to store image using imread() function.
5. Display the image using imshow() function.
6. Add a delay using a waitkey() function.

### • **Code:**

```
# importing cv2
import cv2
# path
path = r'E:\DIP images\Lenna.png'
#Reading an image in default mode
image = cv2.imread(path)
```

```
# Window name in which image is displayed
window_name = 'image'
# Using cv2.imshow() method # Displaying the image
cv2.imshow(window_name, image)
#waits for user to press any key
#(this is necessary to avoid Python kernel form crashing)
cv2.waitKey(0)
#closing all open windows
cv2.destroyAllWindows()
```

- **Output**



**1(b) Aim :**

To create a program to rotate an image.

- **Apparatus use:**

1. A PC with Python 3.7.8 and OpenCV
2. Jupyter notebook

- **Theory**

OpenCV-Python is a library of Python bindings designed to solve computer vision problems. **cv2.rotate()** method is used to rotate a 2D array in multiples of 90 degrees. The function cv2.rotate rotates the array in three different ways.

- **Syntax:**

cv2.cv.rotate( src, rotateCode[, dst] )

**Parameters:**

src: It is the image whose color space is to be changed.

rotateCode: It is an enum to specify how to rotate the array.

dst: It is the output image of the same size and depth as src image. It is an optional parameter.

Return Value: It returns an image.

- **Procedure**

1. Use the cv2. rotate function: Built into OpenCV, but requires constructing a rotation matrix and explicitly applying an affine warp, making the code more verbose.
2. Use the imutils. rotate function: Part of my imutils library.
3. Use the imutils. rotate\_bound function: Also part of my imutils library.

- **Code:**

```
# importing cv2
import cv2
# path
path = r'E:\DIP images\Lena.png'
# Reading an image in default mode
src = cv2.imread(path)
# Window name in which image is displayed
window_name = 'Image'
# Using cv2.rotate() method
# Using cv2.ROTATE_90_CLOCKWISE rotate
# by 90 degrees clockwise
image = cv2.rotate(src, cv2.ROTATE_90_CLOCKWISE)
# Displaying the image
cv2.imshow(window_name, image)
cv2.waitKey(0)
```

- **Output:**



### 1(c) Aim :

To create a program to flip an image.

- **Apparatus use:**

1. A PC with Python 3.7.8 and OpenCV
2. Jupyter notebook

- **Theory**

1. OpenCV-Python is a library of programming functions mainly aimed at real-time computer vision. cv2.
2. flip() method is used to flip a 2D array.
3. The function cv::flip flips a 2D array around vertical, horizontal, or both axes.

- **Syntax:**

**cv2.cv.flip(src, flipCode[, dst] )**

Parameters:

**src:** Input array.

**dst:** Output array of the same size and type as src.

**flip code:** A flag to specify how to flip the array; 0 means flipping around the x- axis and positive value (for example, 1) means flipping around y-axis. Negative value (for example, -1) means flipping around both axes.

**Return Value:** It returns an image.

- **Procedure**

1. Open Python 3.7.8 with Opencv
2. Import cv2.
3. Use the cv2. flip function: Built into OpenCV.
4. Paste a test image in the directory.
5. Create variable to store image using imread() function.
6. Display the image using imshow() function.
7. Add a delay using a waitkey() function.

- **Code:**

```
import cv2
originalImage = cv2.imread('E:\DIP images\Lena.png')
flipVertical = cv2.flip(originalImage, 0)
flipHorizontal = cv2.flip(originalImage, 1)
flipBoth = cv2.flip(originalImage, -1)
cv2.imshow('Original image', originalImage)
cv2.imshow('Flipped vertical image', flipVertical)
cv2.imshow('Flipped horizontal image', flipHorizontal)
cv2.imshow('Flipped both image', flipBoth) cv2.waitKey(0)
cv2.destroyAllWindows()
```

- **Output:**







#### 1(d) Aim :

To create a program to transpose an image.

- **Theory**

OpenCV is a library of programming functions mainly aimed at real-time computer vision. **cv2.transpose()** method is used to transpose a 2D array. The function `cv::transpose` rotate the image 90 degrees Counter clockwise.

- **Syntax:** `cv2.cv.transpose( src[, dst] )`

Parameters:

**src:** It is the image whose matrix is to be transposed.

**dst:** It is the output image of the same size and depth as src image. It is an optional parameter.

**Return Value:** It returns an image.

- **Procedure**

1. Open Python 3.7.8 with Opencv
2. Import cv2.
3. Use the cv2.transpose function: Built into OpenCV.
4. Paste a test image in the directory.

5. Create variable to store image using imread() function.
6. Display the image using imshow() function.
7. Add a delay using a waitkey() function

- **Code:**

```
# importing cv2
import cv2
# path
path = r'E:\DIP images\Lena.png'
# Reading an image in default mode
src = cv2.imread(path)
# Window name in which image is displayed
window_name = 'Image'
# Using cv2.transpose() method
image = cv2.transpose(src)
# Displaying the image
cv2.imshow(window_name, image)
cv2.waitKey(0)
```

- **Output:**

**1(e) Aim :**

To create a program to convert original image to grayscale.

- **Theory**

Grayscale is the process of converting an image from other color spaces e.g. RGB, CMYK, HSV, etc. to shades of gray. It varies between complete black and complete white.

- **Procedure**



1. Open Python 3.7.8 with Opencv
2. Import cv2.
3. Convert an Image to Grayscale in Python Using the **image.convert()** Method of the pillow Library
4. Convert an Image to Grayscale in Python Using the **color.rgb2gray()** Method of the scikit-image Module
5. Convert an Image to Grayscale in Python Using the cv2.imread() Method of the OpenCV Library
6. Convert an Image to Grayscale in Python.
7. Display the image using imshow() function.
8. Add a delay using a waitkey() function

- **Code:**

```
# import opencv
import cv2
# Load the input image
image = cv2.imread('E:\DIP images\Lena.png')
cv2.imshow('Original', image)
cv2.waitKey(0)
# Use the cvtColor() function to grayscale the image
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imshow('Grayscale', gray_image)
cv2.waitKey(0)
# Window shown waits for any key pressing event
cv2.destroyAllWindows()
```

- **Output:**



1. What is the role of OpenCV in digital image processing?
2. Explain how `cv2.imread()` and `cv2.imshow()` functions work.
3. What is the purpose of `cv2.waitKey()` and `cv2.destroyAllWindows()`?
4. What formats of images can OpenCV read and write?
5. How do you resize an image using OpenCV?
6. Demonstrate how to convert a color image to grayscale using OpenCV.
7. Write a code snippet to crop a region from an image.
8. How can you save an image using OpenCV?
9. What will happen if `cv2.waitKey(0)` is not included after `cv2.imshow()`?
10. How does image rotation differ from flipping in OpenCV?
11. How do changes in resolution affect image quality and processing?
12. Imagine your image does not load properly. What debugging steps would you take?
13. How would you optimize image loading in a project with thousands of images?
14. Suggest ways to automate a batch resize of 100+ images using Python and OpenCV.